

Exerciții — Încapsularea

private + getteri/setteri + validare (model: clasa Persoana)

MyCodeSchool

Ce este încapsularea

Încapsularea = ascunzi datele (atributele) în interiorul clasei și le lași accesibile doar prin metode. Nimeni din afară nu atinge direct un câmp.

Trei reguli, exact ca în clasa Persoana din proiect:

1. Atributele sunt private.
2. Accesul din afară se face doar prin metode: **getter** (getX()) și **setter** (setX(...)).
3. Setterul **validează** înainte să scrie. Dacă valoarea e greșită, nu o pune și anunță.

```
public class Persoana {  
    private int varsta; // 1. privat  
  
    public int getVarsta() { // 2. getter  
        return varsta;  
    }  
  
    public void setVarsta(int varsta) { // 2. setter + 3. validare  
        if (varsta >= 0) {  
            this.varsta = varsta;  
        } else {  
            System.out.println("Varsta nu poate fi mai mica de 0.");  
        }  
    }  
}
```

Regula jocului (ca la EXERCITII.md): **citești cerința, scrii singur clasa, apoi o testezi din Main** cu valorile de verificare. Fiecare clasă nouă stă în fișierul ei sub src/app/.

Nivel 1 — Bazele: private + getteri/setteri

Creează clasa Elev cu attributele: nume (String), nota (int), materie (String).

1. Fă attributele private

Toate cele trei attribute trebuie să fie private. Din Main, `elev.nota = 10`; NU trebuie să compileze.

2. Getteri

```
String getNume()  
int getNota()  
String getMaterie()
```

3. Setteri simpli (deocamdată fără validare)

```
void setNume(String nume)  
void setNota(int nota)  
void setMaterie(String materie)
```

4. Constructor complet care folosește setterii

```
public Elev(String nume, int nota, String materie)
```

Constructorul NU scrie direct `this.nota = nota`; ci apelează `setNota(nota)` etc. (exact ca în Persoana). Așa validarea de mai târziu se aplică și la construire.

Verificare: în Main creezi `new Elev("Adi", 9, "Matematica")` și afișezi `getNume()` -> Adi.

Nivel 2 — Validare în setteri

Adaugă validări în clasa Elev. Modelul e setNume / setVarsta / setOras din Persoana.

5. Nota între 1 și 10 (interval)

```
void setNota(int nota)
```

Acceptă doar 1..10. Altfel afișează Nota trebuie sa fie între 1 si 10. și nu modifică.

Verificare: setNota(15) -> mesaj, nota rămâne cea veche. setNota(8) -> nota devine 8.

6. Materie din listă permisă (whitelist)

```
void setMaterie(String materie)
```

Acceptă doar din: Matematica, Informatica, Romana, Engleza (comparație fără majuscule, ca la setOras). Altfel: Materia nu este in lista.

7. Nume interzis (blacklist)

```
void setNume(String nume)
```

Refuză numele din lista neagră: test, anonim, xxx (ca la setNume din Persoana). Altfel afișează Numele este pe lista neagra.

8. Metoda descriere()

```
String descriere()
```

Întoarce un text cu toate cele trei câmpuri, câte unul pe linie (ca descriere() din Persoana și Masina). **Atenție:** în Persoana, descriere() e private — deci nu poate fi apelată din Main. Aici fă-o public ca să o poți testa.

Nivel 3 — Constructori supraîncărcați

Creează clasa ContBancar cu: titular (String, privat), sold (double, privat).

9. Trei constructori (supraîncărcare)

```
public ContBancar()                // gol
public ContBancar(String titular)  // doar titular, sold 0
public ContBancar(String titular, double soldInitial)
```

Ca în Persoana, fiecare constructor apelează setterii, nu scrie direct câmpurile.

10. sold are DOAR getter, fără setter public

```
double getSold()
```

Soldul nu se schimbă direct din afară — se schimbă doar prin depune / retrage (Nivel 4). Țasta e miezul încapsulării: protejezi o valoare de modificări necontrolate.

11. setTitular cu validare

```
void setTitular(String titular)
```

Refuză null și string gol (""). Altfel: Titular invalid.

Nivel 4 — Comportament peste stare validată

Tot în ContBancar. Aici setterul lipsește intenționat — logica trece prin metode.

12. Depunere

```
void depune(double suma)
```

Adaugă suma la sold, dar doar dacă suma > 0. Altfel: Suma depusa trebuie să fie pozitivă.

Verificare: cont cu sold 100, depune(50) -> getSold() = 150. depune(-20) -> mesaj, sold rămâne 150.

13. Retrageri

```
void retrage(double suma)
```

Scade suma din sold, doar dacă: suma > 0 **ȘI** suma <= sold (fonduri suficiente). Altfel: Fonduri insuficiente sau suma invalidă.

Verificare: sold 150, retrage(200) -> mesaj, sold rămâne 150. retrage(50) -> sold 100.

14. descriere() pentru cont

```
String descriere()
```

Întoarce Titular: <nume> și Sold: <sold> pe două linii.

Nivel 5 — Refactor: Mașina cu încapsulare

Clasa ta Mașina are acum atributele public (fără încapsulare). Fă o copie încapsulată — clasă nouă MașinaEnc — ca să nu strici MașinaService.

15. Toate atributele private

marca, model, kilometraj, pret, anFabricatie, modTransmisie -> private.

16. Getteri pentru toate

Câte un getX() pentru fiecare atribut.

17. Setteri cu validare

```
void setPret(int pret)           // pret > 0
void setKilometraj(int km)       // km >= 0
void setAnFabricatie(int an)     // intre 1950 si 2025
void setModTransmisie(String t)  // doar "Manuala" sau "Automata"
```

marca și model doar să nu fie null / goale.

18. Constructor complet prin setteri

```
public MasinaEnc(String marca, String model, int kilometraj, int pret, int anFabricatie,
```

Verificare: new MasinaEnc("Audi", "A4", 12000, 18000, 1993, "Manuala") merge; new MasinaEnc("Audi", "A4", -5, 0, 1800, "Zbor") -> 3 mesaje de validare, câmpurile respective rămân neinițializate (0 / null).

Nivel 6 — Provocări

19. Cont cu limită de descoperire (invariant)

Adaugă în `ContBancar` un câmp `private double limitaDescoperire` (cât poate coborî soldul sub 0). Modifică `retrage` să accepte până la `sold - suma >= -limitaDescoperire`. Câmpul se setează doar prin constructor sau un setter care refuză valori negative.

20. Clasă imutabilă

Creează `Coordonata` cu `private final double lat` și `private final double lon`. Doar constructor + getteri, **fără setteri**. Odată creată, nu se mai poate schimba. Adaugă `String descriere()` care le afișează.

21. Validare compusă

În `Elev`, adaugă `boolean estePromovat()` care întoarce `true` doar dacă `nota >= 5` **ȘI** materie nu e `null`. Folosește doar getteri interni, nu accesa câmpurile direct.

22. De ce contează încapsularea (scris, nu cod)

Scrie 2–3 fraze ca răspuns, într-un comentariu în `Main`: *ce s-ar putea strica dacă sold din ContBancar ar fi public în loc de private?* (Gândește-te la un cod care face `cont.sold = -9999`; fără să treacă prin `retrage`.)